

Retrieval or Fine-Tuning?

Parameter-Efficient Adaptation of Process Reward Models to Financial Numerical Reasoning

Authors to be inserted

Research proposal and implementation plan

Abstract

Process reward models (PRMs) evaluate intermediate reasoning steps and can guide test-time selection more precisely than outcome-only verifiers. Their reliability under domain shift, however, remains uncertain. This project studies whether a general mathematical process verifier can be adapted to financial numerical reasoning through retrieval augmentation, parameter-efficient supervised fine-tuning, or their combination. We derive automatically verifiable step-level supervision from FinQA’s expert-authored questions and executable gold programs. The required core system is a binary discriminative PRM that scores a candidate structured operation conditioned on the financial context, question, and a correct program prefix. Four conditions—unadapted, retrieval-only, LoRA-only, and retrieval plus LoRA—are compared under standard and controlled out-of-distribution splits. Evaluation prioritizes intrinsic process verification, including macro-F1, precision–recall performance, calibration, and first-error localization; a bounded Best-of- N experiment measures downstream utility. A local “financial reasoning checker” demonstrates the verifier without presenting it as an investment-advice system. The design is scoped for two researchers over four weeks, with GPU-based QLoRA training and laptop-local inference. Error typing and generative critiques are explicitly optional extensions attempted only after the binary core is complete.

1 Introduction

Large language models can produce correct-looking answers through invalid intermediate reasoning. Outcome reward models score complete solutions and therefore cannot reliably distinguish a sound derivation from a lucky final answer. PRMs instead assign rewards at intermediate steps, enabling fine-grained error detection and reward-guided selection [5, 10].

Most established PRMs are trained and evaluated on general mathematical reasoning. Financial question answering introduces a controlled but meaningful distribution shift: the arithmetic is familiar, while the inputs contain tables, reporting periods, domain terminology, percentages, and references to previously computed values. FinQA is suitable for studying this shift because it

pairs expert-authored questions over financial reports with supporting evidence, numerical answers, and executable reasoning programs [1].

The central question is not whether another model can be fine-tuned to answer FinQA. Instead, it is whether a process verifier can determine if a *candidate next operation* is valid. We compare two adaptation mechanisms. Retrieval provides similar verified examples at inference time without changing model parameters, following work on retrieval-enhanced PRM generalization [15]. Low-rank adaptation (LoRA) modifies a small set of trainable parameters while freezing the backbone [3]. Their direct comparison under a shared data and compute budget is practically relevant and not answered by simply evaluating a finance-tuned answer generator.

Expected contributions. This project aims to contribute:

1. a reproducible transformation from FinQA programs into positive and hard-negative process-verification examples;
2. a controlled comparison of retrieval, LoRA, and combined PRM adaptation;
3. an evaluation protocol separating step verification from final-answer selection and testing multiple forms of distribution shift; and
4. a laptop-local demonstrator that highlights the first suspicious operation in a structured financial derivation.

2 Background and Related Work

2.1 Automatic process supervision

Math-Shepherd estimates step quality through continuations and final-answer verification, enabling process supervision without human step labels [10]. AutoPSV instead derives automatic process annotations from relative changes in the confidence of an outcome-supervised verifier [6]. These approaches make process supervision scalable, but the quality of automatically inferred labels remains a central concern. The Qwen PRM study reports that naive Monte Carlo labeling can generalize poorly and advocates consensus filtering and combined step- and response-level evaluation [13].

Our core design avoids expensive rollout-based labels wherever possible. It uses executable FinQA programs to construct locally checkable candidate operations. This provides a high-precision experimental substrate while retaining a realistic domain shift.

2.2 What a process reward should represent

A conventional PRM predicts binary correctness for each step. Other work treats a partial solution as a value-estimation problem. OVM learns the potential of a reasoning prefix from outcome supervision [12], while the Process Q-value Model optimizes rankings between candidate steps rather than independent correctness labels [4]. These formulations are valuable but add objectives and sampling requirements. For a one-month project, binary next-operation verification is the required task. Ranking-based rewards are left as future work.

2.3 Generative, error-aware, and reflective PRMs

Generative PRMs reason before judging rather than outputting only a scalar. GenPRM uses explicit reasoning and code verification and scales verification-time computation [14]; R-PRM combines rationale-generating supervision with preference optimization [8]. Error-aware hierarchical supervision predicts specific error types before estimating correctness [7]. Reflective-PRM research further challenges the common assumption that all steps after the first error are necessarily invalid, since long chains can contain self-correction [11].

These findings motivate optional critique generation, but they also argue for a conservative core task. We evaluate a candidate next operation after a known-correct prefix, so a negative label does not contaminate an entire suffix. If time remains, the binary head will be extended with error-type labels or short critiques.

2.4 Generalization and evaluation

RetrievalPRM identifies both question-level and step-level out-of-distribution (OOD) shifts and retrieves related questions and steps as demonstrations [15]. PRM-Bench shows that aggregate correctness alone does not characterize process verification; it evaluates fine-grained soundness and sensitivity across diverse errors [9]. Best-of- N can also reward outcome recognition rather than genuine process sensitivity [13]. We therefore make intrinsic step-level evaluation primary and treat Best-of- N as secondary evidence.

Table 1: Notation used in the process-verification task. A star marks gold, dataset-provided reasoning; a tilde marks a candidate that must be judged.

Symbol	Meaning
E	Financial evidence: the relevant table rows and text.
q	The FinQA question.
P^*	The complete gold executable program.
T	Number of operations in the gold program.
t	Position of the operation currently being evaluated.
s_t^*	Correct gold operation at position t .
$P_{<t}^*$	Correct gold operations before position t .
$\tilde{s}_t$	Candidate next operation judged by the PRM.
r_θ	PRM score; larger values mean “more likely correct.”
θ	Trainable model parameters, including the LoRA adapter.
y	Binary target: 1 for correct and 0 for incorrect.
x_t^+	One positive training example constructed at position t .

3 Problem Definition

Let a FinQA instance contain financial evidence E , a question q , and an executable gold program

$$P^* = (s_1^*, s_2^*, \dots, s_T^*), \quad (1)$$

where each s_t^* is an operator with arguments drawn from E , constants, or prior results. At step t , the verifier receives a correct prefix $P_{<t}^*$ and a candidate operation $\tilde{s}_t$. It predicts

$$r_\theta(E, q, P_{<t}^*, \tilde{s}_t) \in [0, 1], \quad (2)$$

interpreted as the probability that $\tilde{s}_t$ is a valid next operation.

The required model minimizes binary cross-entropy:

$$\mathcal{L}_{\text{BCE}} = -y \log r_\theta - (1 - y) \log(1 - r_\theta), \quad (3)$$

where $y = 1$ for a gold step and $y = 0$ for a verified corruption. This formulation tests process verification rather than direct answer generation.

4 Research Questions and Hypotheses

RQ1: How much do retrieval augmentation and LoRA-based SFT improve a general verifier’s accuracy on structured FinQA operations?

RQ2: Do retrieval and LoRA provide complementary gains, or does their combination yield little improvement over the stronger individual method?

RQ3: How robust are the adaptation methods to company, program-template, reasoning-length, and generator-style shifts?

RQ4: Do improvements in intrinsic step verification translate into better selection of complete candidate programs?

H1: Both retrieval and LoRA will outperform the unadapted verifier on in-domain step classification.

H2: LoRA will perform best on frequent in-domain operation patterns, while retrieval will lose less performance under template and company shift.

H3: Retrieval plus LoRA will achieve the strongest mean performance but may be less calibration-stable if retrieved examples are misleading.

H4: Step-level gains will correlate only imperfectly with Best-of- N gains, confirming that response-level selection is an incomplete PRM evaluation.

5 Dataset Construction

5.1 What FinQA provides and what we create

FinQA does *not* contain ready-made positive and negative examples for training a binary PRM. For each source instance, FinQA provides the financial report context, a question, supporting evidence, the numerical answer, and one expert-annotated executable gold program [1]. The gold program contains correct operations, but the dataset does not provide a matched set of plausible incorrect next operations.

FinQA contains 8,281 question-answer instances drawn from 2,789 financial-report pages: 6,251 training, 883 development, and 1,147 test instances. The reports do not overlap across these splits. Across the full dataset, 59.10% of gold programs contain one operation, 32.71% contain two, and 8.19% contain three or more [1]. Program operations—rather than source questions—determine the number of positive PRM examples.

Our preprocessing pipeline constructs the PRM dataset. It splits each gold program into individual operations, uses each gold operation as a positive candidate, and creates plausible negative candidates by modifying that operation. Each constructed example stores the original FinQA identifier, the evidence and question, the correct prefix, the candidate next operation, its binary label, and the rule used to create it. Thus, FinQA supplies the source problems and correct programs; this project supplies the step-level classification examples.

5.2 Source data and leakage controls

We use the official FinQA splits and preserve document-level separation where supplied [1]. The test set is never used for retrieval, threshold selection, prompt development, or negative-generation tuning. Retrieval indexes contain training examples only. Development data is used for model selection and calibration.

FinQA’s program accuracy can penalize alternative valid programs, whereas execution accuracy can accept

accidental equivalences. We therefore do not label a candidate step as incorrect simply because it differs from the gold program. A negative label is used only when the candidate is clearly wrong—for example, it uses the wrong operator, the wrong input value, or an invalid intermediate-result reference. Any modified step that could still be valid is discarded.

5.3 Positive examples

For every operation s_t^* in a gold program, the pipeline creates one positive example:

$$x_t^+ = (E, q, P_{<t}^*, s_t^*, 1). \quad (4)$$

In plain language, this tuple contains the evidence, the question, all correct earlier operations, the correct next operation, and the label 1 (“correct”). For a three-operation gold program, the pipeline therefore creates three positive examples, one for each position. Evidence is limited to the gold supporting text/table facts in the initial experiments. A later robustness experiment may add distractor evidence.

5.4 Hard-negative generation

FinQA does not provide negative steps, so the pipeline creates them from the positive examples. For each correct operation, it makes one controlled change and labels the modified candidate 0 only if the validator confirms that the change is invalid. The retained mutation types are:

- **operator substitution:** replace an operator with a type-compatible alternative;
- **operand reversal:** swap arguments for non-commutative operations;
- **evidence substitution:** replace an argument with another number from the same table or text;
- **reference corruption:** use the wrong prior intermediate result;
- **constant corruption:** alter percentage scales or fixed constants;
- **result corruption:** retain the operation but provide an inconsistent computed value; and
- **premature termination:** predict a final answer before required dependencies are resolved.

Each corruption is passed through a validator. Commutative-equivalent or execution-equivalent mutations are discarded rather than labeled negative. A stratified manual audit estimates residual label noise for every corruption class. The audit size will be fixed before model evaluation; the initial target is at least 50 examples per retained error class or all examples for smaller classes.

Design rationale and precedent. The mutations follow four principles. First, they change only one element at a time, so the error label has a clear cause. Second, replacement operators and operands remain type-compatible and are drawn from plausible local

evidence, preventing trivial negatives. Third, executable checks reject mutations that remain mathematically valid. Fourth, sampling is balanced across error types so the verifier cannot succeed by learning one frequent artifact.

Controlled error injection is established in PRM evaluation: PRMBench modifies reasoning chains to introduce targeted errors and evaluates whether verifiers detect them [9]. Error-aware PRMs likewise benefit from explicitly labeled mathematical and consistency errors [7]. However, our exact procedure is not copied from those papers. The proposed contribution is a deterministic version specialized to FinQA’s executable operations. Unlike LLM-generated corruptions, it records the changed operator or argument and checks the resulting program mechanically. A small evaluation on naturally produced model errors is retained to test whether performance transfers beyond synthetic mutations.

Listing 1: Conceptual construction of a next-step example.

```
context: financial text and table evidence
question: "What was the percentage increase?"
prefix: subtract(120, 100) -> #0
candidate: divide(#0, 120) -> #1
label: incorrect
error: wrong denominator
```

5.5 Class balance and sampling

Using FinQA’s published program-length distribution, the 6,251 training questions imply approximately 9,300 or more gold operations, and therefore at least that many positive candidates; the exact number will be computed after parsing and validation. The target is two accepted hard negatives per positive, giving roughly 18,600 negatives and about 28,000 total training examples. If a gold step has fewer than two unambiguously invalid mutations, it contributes fewer negatives rather than accepting a questionable label.

Development and test examples are created independently from their official source splits and never enter training or the retrieval index. For primary evaluation, each gold operation is paired with one accepted negative sampled according to a fixed error-type quota, producing a balanced test set. A second evaluation retains all accepted negatives to measure performance under the constructed natural distribution. All final counts, rejection rates, and counts by mutation type will be reported after preprocessing rather than treated as predetermined results.

Training batches will balance positive and negative labels. Negative types will be capped to prevent simple operator substitutions from dominating. A 5,000-example pilot is used to debug the pipeline and LoRA configuration before the full approximately 28,000-example training run.

Table 2: Required comparison. Only the adapter parameters differ between base and LoRA conditions.

Condition	Retrieval	LoRA-SFT
Base PRM	No	No
Retrieval PRM	Yes	No
FinQA LoRA-PRM	No	Yes
Retrieval + LoRA	Yes	Yes

6 Experimental Conditions

All conditions use the same underlying verifier backbone, input serialization, candidate steps, and evaluation sets. The backbone will be a recent open-weight model in the 1.5B–4B range. The exact checkpoint will be frozen before experiments based on license compatibility, tokenizer support for FinQA notation, and successful local quantized inference.

6.1 Base verifier

The base condition receives a fixed instruction and the serialized example. It outputs a probability or two label logits for **correct** and **incorrect**. Prompt wording and decision parsing are fixed on development data.

6.2 Retrieval PRM

Training examples are embedded and indexed. At inference, the system retrieves k demonstrations using (a) question similarity and (b) joint similarity between the question, prefix, and candidate operation. We evaluate small k values that fit local context limits. Random retrieval and no retrieval are controls. Retrieval labels are never taken from evaluation examples.

6.3 LoRA-PRM

The backbone is frozen and trained with LoRA adapters on the binary process dataset. Hyperparameters are selected from a small predefined grid on development macro-F1. QLoRA may be used to reduce memory requirements [2]. Training runs on Colab or a university GPU; the final adapter or merged quantized model must run on the 16-GB Apple-silicon laptop.

6.4 Combined condition

The LoRA-adapted verifier receives demonstrations from the same retrieval pipeline. This isolates whether contextual and parameter adaptation are complementary. The combined prompt length and inference cost are reported.

7 Controlled Distribution Shifts

The following evaluations address different notions of generalization:

1. **Official split:** standard FinQA train/development/test evaluation.
2. **Company OOD:** hold out companies from adaptation and retrieval demonstrations.
3. **Template OOD:** group programs by normalized operator sequence and hold out selected compositions.
4. **Length OOD:** train/adapt on shorter programs and test on longer programs.
5. **Generator-style OOD:** optional evaluation on structured candidates produced by a model rather than deterministic corruptions.

The first four are required. Generator-style OOD is attempted only if candidate parsing is reliable by the end of Week 2.

8 Evaluation Protocol

8.1 Intrinsic process-verification metrics

The primary metric is macro-F1, which weights correct and incorrect classes equally. We additionally report:

- precision, recall, and per-class F1;
- area under the ROC and precision–recall curves;
- Brier score and expected calibration error;
- first-error localization accuracy on corrupted full programs; and
- accuracy by corruption type, program length, and OOD split.

Thresholds are selected once on development data. Because all systems evaluate the same examples, uncertainty is estimated through paired bootstrap confidence intervals. Pairwise differences in binary predictions may additionally be tested using McNemar’s test, with Holm correction for the planned comparisons.

8.2 Downstream Best-of- N evaluation

A bounded downstream experiment constructs $N \in \{4, 8\}$ complete candidate programs per question. Initially, candidates are generated through controlled program corruption; model-generated programs may be added if time permits. We compare random selection, execution-equivalent majority voting where defined, the four verifier conditions, and an oracle that selects a correct candidate whenever one exists.

Step scores will be aggregated using predefined rules: minimum step score, mean log score, and length-normalized sum of log scores. Reporting multiple aggregations is necessary because products favor short programs and minima can be dominated by one miscalibrated step.

The main downstream metric is selection accuracy conditional on at least one correct candidate. Final execution accuracy and oracle gap closed are secondary. No claim about PRM quality will be based on Best-of- N alone.

Table 3: Planned primary results. Values are intentionally blank until experiments are run.

Method	Macro-F1	AUPRC	Brier↓	First error
Base	—	—	—	—
Retrieval	—	—	—	—
LoRA	—	—	—	—
Retrieval + LoRA	—	—	—	—

8.3 Planned result presentation

No numerical findings are assumed in this proposal. The final report will include a table of the following form:

9 Implementation Plan

9.1 System architecture

The implementation consists of six modules:

1. **FinQA loader:** reads official splits, evidence, answers, and programs;
2. **program executor:** validates operations and intermediate references;
3. **process-data builder:** creates positives, corruptions, provenance, and audit samples;
4. **retriever:** embeds training examples and returns controlled demonstrations;
5. **verifier:** runs the base or LoRA-adapted binary PRM; and
6. **evaluation/UI:** calculates metrics and presents step-level judgments.

Every generated example receives a stable identifier linking it to the FinQA source instance, program step, corruption rule, random seed, and validator version. Configuration files record model revisions, prompts, LoRA hyperparameters, retrieval settings, and splits.

9.2 Input serialization

A uniform text schema will be used across all conditions:

```
[EVIDENCE]
<supporting text and normalized table rows>
[QUESTION]
<question>
[CORRECT PREFIX]
<operations and intermediate values>
[CANDIDATE NEXT OPERATION]
<candidate>
[TASK]
Return CORRECT or INCORRECT.
```

Intermediate values are executed rather than trusted from generated text. Serialization variants are compared only on development data and then frozen.

9.3 Compute and deployment

Training uses Colab or a university GPU. Mixed precision, gradient accumulation, and QLoRA are permitted. The required deliverable is laptop-local inference using

a quantized model runtime. Latency, peak memory, model size, and retrieval-index size are reported alongside accuracy. A 1.5B checkpoint is the fallback if a 4B checkpoint cannot meet the memory or latency target.

9.4 Reproducibility

The repository will include environment files, fixed seeds, dataset checksums, preprocessing scripts, configuration files, and a one-command evaluation entry point. At least two training seeds are desirable; if compute permits only one full training run, the limitation will be stated and bootstrap uncertainty will not be misrepresented as training-seed variance.

10 Demonstrator

The local demonstrator is a financial numerical reasoning checker, not an investment system. A user can select a FinQA example or enter a compatible small table, question, correct-prefix assumption, and candidate operation. The interface displays:

- the binary process reward;
- the first operation below a calibrated threshold;
- retrieved examples and their provenance; and
- deterministic execution details where available.

The interface must distinguish model judgment from executable verification. It must not produce stock recommendations, expected returns, or personalized financial advice.

11 Optional Extensions and Stop Rule

The following extensions are attempted only after all four core conditions have complete intrinsic results and the evaluation code is frozen:

1. **Error-type head:** predict the corruption category jointly with correctness.
2. **Generative critique:** explain the detected error before returning the binary label.
3. **Natural-language steps:** verbalize programs or evaluate model-generated reasoning traces.
4. **Pairwise ranking:** compare candidate next operations using a Q-value-style objective.

The first optional priority is error typing because labels already arise from the corruption generator. Generative critiques are second because they require additional target construction and evaluation.

12 Risks and Mitigations

Synthetic negatives are too easy. A classifier may learn mutation artifacts rather than validity. We include same-table substitutions, type-compatible operators, naturally generated candidates where feasible, and

evaluation by corruption type.

Alternative programs are mislabeled. We discard execution-equivalent candidates and manually audit ambiguous templates. The report separates exact program agreement from semantic validity.

Retrieval leaks labels. Only training examples enter the index; company and template OOD indexes exclude held-out groups. Duplicate and near-duplicate checks are logged.

LoRA training exceeds compute. We begin with a small subset and 1.5B fallback, cap sequence length, and prioritize one reliable model over multiple backbones.

Best-of- N rewards outcome sensitivity. Intrinsic step metrics remain primary, and counterexamples with correct answers but corrupted processes are analyzed.

The binary system is not sufficiently informative. Error typing is the first optional extension, but the binary verifier remains a complete research contribution if carefully evaluated.

13 Ethical and Practical Considerations

FinQA is a research benchmark derived from public financial reports, not a source of current prices or personalized investor information. The demonstrator is intended to audit structured numerical reasoning. It must communicate uncertainty, cite dataset provenance, avoid claims of guaranteed correctness, and never convert a process score into a buy/sell recommendation. Model outputs can be wrong even when well calibrated; deterministic execution should take precedence for operations that can be checked exactly.

14 Success Criteria

The project is considered complete if it delivers:

1. a validated FinQA-to-process-supervision pipeline;
2. all four conditions evaluated on the official split and at least three controlled shifts;
3. intrinsic metrics with uncertainty and a bounded Best-of- N study;
4. a reproducible LoRA adapter and laptop-local inference path; and
5. a working demonstrator that exposes step scores and retrieved provenance.

Success does not require LoRA or retrieval to improve performance. A carefully supported negative result—for example, that retrieval matches LoRA or that synthetic adaptation fails on natural candidates—would still answer the research questions.

15 Conclusion

This proposal turns FinQA into a controlled testbed for process-reward-model adaptation. The staged design

makes binary next-operation verification the required core, compares contextual and parameter adaptation fairly, and evaluates process sensitivity independently of final-answer selection. It is feasible for two researchers in one month because it uses existing expert-authored data, executable programs, parameter-efficient training, and bounded downstream experiments. At the same time, its OOD splits, calibration analysis, and retrieval–LoRA comparison provide a substantive research question beyond straightforward benchmark fine-tuning.

References

- [1] Zhiyu Chen, Wenhui Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. Finqa: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3697–3711, 2021. doi: 10.18653/v1/2021.emnlp-main.300.
- [2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*, 2023.
- [3] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [4] Wendi Li and Yixuan Li. Process reward model with q-value rankings. In *International Conference on Learning Representations*, 2025.
- [5] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- [6] Jianqiao Lu, Zhiyang Dou, Hongru Wang, Zeyu Cao, Jianbo Dai, Yingjia Wan, and Zhijiang Guo. Autopsv: Automated process-supervised verifier. In *Advances in Neural Information Processing Systems*, 2024.
- [7] Tej Deep Pala, Panshul Sharma, Amir Zadeh, Chuan Li, and Soujanya Poria. Error typing for smarter rewards: Improving process reward models with error-aware hierarchical supervision. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.
- [8] Shuaijie She, Junxiao Liu, Yifeng Liu, Jiajun Chen, Xin Huang, and Shujian Huang. R-prm: Reasoning-driven process reward modeling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.
- [9] Mingyang Song, Zhaochen Su, Xiaoye Qu, Jiawei Zhou, and Yu Cheng. Prmbench: A fine-grained and challenging benchmark for process-level reward models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025.
- [10] Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Y. Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 9426–9439, 2024.
- [11] Zhaohui Yang, Chenghua He, Xiaowen Shi, Linjing Li, Qiyue Yin, Shihong Deng, and Daxin Jiang. Beyond the first error: Process reward models for reflective mathematical reasoning. *arXiv preprint arXiv:2505.14391*, 2025.
- [12] Fei Yu, Anningzhe Gao, and Benyou Wang. Ovm, outcome-supervised value models for planning in mathematical reasoning. *arXiv preprint arXiv:2311.09724*, 2023.
- [13] Zhenru Zhang, Chujie Zheng, Yangzhen Wu, Beichen Zhang, Runji Lin, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. The lessons of developing process reward models in mathematical reasoning. *arXiv preprint arXiv:2501.07301*, 2025.
- [14] Jian Zhao, Runze Liu, Kaiyan Zhang, Zhimu Zhou, Junqi Gao, Dong Li, Jiafei Lyu, Zhouyi Qian, Biqing Qi, Xiu Li, and Bowen Zhou. Genprm: Scaling test-time compute of process reward models via generative reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. First released as arXiv:2504.00891 in 2025.
- [15] Jiachen Zhu, Congmin Zheng, Jianghao Lin, Kounianhua Du, Ying Wen, Yong Yu, Jun Wang, and Weinan Zhang. Retrieval-augmented process reward model for generalizable mathematical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.